



早稲田大学
WASEDA University

WASEDA University

Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫 鶴鳴

SUN Heming

Apr. 28th



Outline

- ◆ Review
- ◆ Exercise using for/while loop
- ◆ Precedence

Review

◆ If-Else

If-Else

else if

turnoff for other directions

```
if ( expression1 ) {  
    statement  
} else if ( expression2 ) {  
    statement  
} ...  
...  
} else if ( expressionp ) {  
    statement  
} else {  
    statement  
}
```



BMI	Result
<18.5	Underweight
18.5-25	Normal
25-30	Overweight
>30	Obese

```
if (bmi < 18.5) {  
    printf("Underweight¥n");  
} else if (bmi < 25) {  
    printf("Normal¥n");  
} else if (bmi < 30) {  
    printf("Overweight¥n");  
} else {  
    printf("Obese¥n");  
}
```

Review (cont.)

◆ While loop

While

- Loop- While

```
while ( expression ) {  
    statement1;  
    statement2;  
    ...  
    statementn;  
}
```

If the expression is true,
statement1, statement2, ... statementn
will be executed.

Review (cont.)

◆ For loop

a method to define a loop as while.

```
for ( initialization_expression; loop_condition; increment_expression) {  
    statement1;  
    statement2;  
    ...  
    statementn;  
}
```



- initialization_expression is executed before execution of the loop starts.
- The execution of the loop continues until the loop_condition is false.
- The increment_expression is executed at the end of each loop iteration.

Outline

- ◆ Review
- ◆ Exercise using for/while loop
- ◆ Precedence

Exercise1

◆ Find the prime numbers among 2 – 20

✦ Write program

- Prime numbers 2, 3, 5, 7, 11, 13, 17, 19
 - 3/2 : OK ... 3
 - 4/2 : NG
 - 5/2 : OK
 - 5/3 : OK
 - 5/4 : OK ... 5
- gcc – primenumber.c
- ./a.out

Exercise1 (cont.)

```
#include <stdio.h>

int main()
{
    for(int num = 2; num < 20; num++)
    {
        int prime_flag = 1;
        for(int div = 2; div < num; div++ )
        {
            if( (num%div)==0)
            {
                prime_flag = prime_flag && 0;
            }
        }

        if(prime_flag == 1)
            printf("%d \n", num);
    }

    return 0;
}
```


Exercise2

- ◆ Print the matrix table as shown in the following

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Exercise2 (cont.)

```
#include <stdio.h>

int main()
{
    for(int i=1;i<10;i++)
    {
        for(int j=1;j<10;j++)
        {
            printf("%d ", i*j);
        }
        printf("\n");
    }

    return 0;
}
```

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

The output isn't very pretty because the numbers are not right-justified. That's easy to fix; if we augment each %d in the printf statement with a width, the numbers printed will be right-justified in their fields.

Exercise2 (cont.)

%d	print as decimal integer
%6d	print as decimal integer, at least 6 characters wide
%f	print as floating point
%6f	print as floating point, at least 6 characters wide
%.2f	print as floating point, 2 characters after decimal point
%6.2f	print as floating point, at least 6 wide and 2 after decimal point

Exercise2 (cont.)

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Exercise2 (cont.)

```
#include <stdio.h>

int main()
{
    for(int i=1;i<10;i++)
    {
        for(int j=1;j<10;j++)
        {
            printf("%3d ", i*j);
        }
        printf("\n");
    }

    return 0;
}
```

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Outline

- ◆ Review
- ◆ Exercise using for/while loop
- ◆ Precedence

Precedence

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){list}	Compound literal(C99)	
2	++ --	Prefix increment and decrement	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Type cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof _Alignof	Size-of Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	?:	Ternary conditional	Right-to-Left
14	=	Simple assignment	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= =	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

Example 1

◆ Try the following example to understand operator precedence in C

```
#include <stdio.h>

main() {

    int a = 20;
    int b = 10;
    int c = 15;
    int d = 5;
    int e;

    e = (a + b) * c / d;    // ( 30 * 15 ) / 5
    printf("Value of (a + b) * c / d is : %d\n", e);

    e = ((a + b) * c) / d;  // (30 * 15) / 5
    printf("Value of ((a + b) * c) / d is : %d\n", e);

    e = (a + b) * (c / d);  // (30) * (15/5)
    printf("Value of (a + b) * (c / d) is : %d\n", e);

    e = a + (b * c) / d;    // 20 + (150/5)
    printf("Value of a + (b * c) / d is : %d\n", e);

    return 0;
}
```

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){list}	Compound literal(C99)	
2	++ --	Prefix increment and decrement	Right-to-left
	+-	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Type cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of	
	_Alignof	Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+-	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	?:	Ternary conditional	Right-to-Left
14	=	Simple assignment	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= =	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

Example 2

◆ Try the following example to understand operator precedence in C

❖ Think it by yourself

❖ Verify the result by programming

$(1 > 2 + 3 \ \&\& \ 4)$

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){list}	Compound literal(C99)	
2	++ --	Prefix increment and decrement	Right-to-left
	+-	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Type cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of	
	_Alignof	Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+-	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	?:	Ternary conditional	Right-to-Left
14	=	Simple assignment	Right-to-Left
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
15	&= ^= =	Assignment by bitwise AND, XOR, and OR	Left-to-right
	,	Comma	

Example 2 (cont.)

`(1 > 2 + 3 && 4)`

This expression is equivalent to:

`((1 > (2 + 3)) && 4)`

i.e., `(2 + 3)` executes first resulting into 5
 then, first part of the expression `(1 > 5)`
 executes resulting into 0 (false)
 then, `(0 && 4)` executes resulting into 0
 (false)

Precedence	Operator	Description	Associativity
1	<code>++ --</code>	Suffix/postfix increment and decrement	Left-to-right
	<code>()</code>	Function call	
	<code>[]</code>	Array subscripting	
	<code>.</code>	Structure and union member access	
	<code>-></code>	Structure and union member access through pointer	
	<code>(type){list}</code>	Compound literal(C99)	
2	<code>++ --</code>	Prefix increment and decrement	Right-to-left
	<code>+-</code>	Unary plus and minus	
	<code>! ~</code>	Logical NOT and bitwise NOT	
	<code>(type)</code>	Type cast	
	<code>*</code>	Indirection (dereference)	
	<code>&</code>	Address-of	
	<code>sizeof</code>	Size-of	
	<code>_Alignof</code>	Alignment requirement(C11)	
3	<code>* / %</code>	Multiplication, division, and remainder	Left-to-right
4	<code>+-</code>	Addition and subtraction	
5	<code><< >></code>	Bitwise left shift and right shift	
6	<code>< <=</code>	For relational operators < and ≤ respectively	
	<code>> >=</code>	For relational operators > and ≥ respectively	
7	<code>== !=</code>	For relational = and ≠ respectively	
8	<code>&</code>	Bitwise AND	
9	<code>^</code>	Bitwise XOR (exclusive or)	
10	<code> </code>	Bitwise OR (inclusive or)	
11	<code>&&</code>	Logical AND	
12	<code> </code>	Logical OR	
13	<code>?:</code>	Ternary conditional	Right-to-Left
14	<code>=</code>	Simple assignment	
	<code>+= -=</code>	Assignment by sum and difference	
	<code>*= /= %=</code>	Assignment by product, quotient, and remainder	
	<code><<= >>=</code>	Assignment by bitwise left shift and right shift	
	<code>&= ^= =</code>	Assignment by bitwise AND, XOR, and OR	
15	<code>,</code>	Comma	Left-to-right

Example 3

- ◆ Try the following example to understand operator precedence in C

1 == 2 != 3

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){list}	Compound literal(C99)	
2	++ --	Prefix increment and decrement	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Type cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of	
	_Alignof	Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
		For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	?:	Ternary conditional	Right-to-Left
14	=	Simple assignment	Right-to-Left
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= =	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

Example 3 (cont.)

`1 == 2 != 3`

Here, operators `==` and `!=` have same precedence. The associativity of both `==` and `!=` is left to right

i.e., the expression on the left is executed first and moves towards the right.

Thus, the expression above is equivalent to :

`((1 == 2) != 3)`

i.e., `(1 == 2)` executes first resulting into 0 (false)
then, `(0 != 3)` executes resulting into 1 (true)

Precedence	Operator	Description	Associativity
1	<code>++ --</code>	Suffix/postfix increment and decrement	Left-to-right
	<code>()</code>	Function call	
	<code>[]</code>	Array subscripting	
	<code>.</code>	Structure and union member access	
	<code>-></code>	Structure and union member access through pointer	
	<code>(type){list}</code>	Compound literal(C99)	
2	<code>++ --</code>	Prefix increment and decrement	Right-to-left
	<code>+-</code>	Unary plus and minus	
	<code>! ~</code>	Logical NOT and bitwise NOT	
	<code>(type)</code>	Type cast	
	<code>*</code>	Indirection (dereference)	
	<code>&</code>	Address-of	
	<code>sizeof</code>	Size-of	
	<code>_Alignof</code>	Alignment requirement(C11)	
3	<code>* / %</code>	Multiplication, division, and remainder	Left-to-right
4	<code>+-</code>	Addition and subtraction	
5	<code><< >></code>	Bitwise left shift and right shift	
6	<code>< <=</code>	For relational operators <code><</code> and <code><=</code> respectively	
	<code>> >=</code>	For relational operators <code>></code> and <code>>=</code> respectively	
7	<code>== !=</code>	For relational <code>=</code> and <code>≠</code> respectively	
8	<code>&</code>	Bitwise AND	
9	<code>^</code>	Bitwise XOR (exclusive or)	
10	<code> </code>	Bitwise OR (inclusive or)	
11	<code>&&</code>	Logical AND	
12	<code> </code>	Logical OR	
13	<code>?:</code>	Ternary conditional	Right-to-Left
14	<code>=</code>	Simple assignment	Right-to-Left
	<code>+= -=</code>	Assignment by sum and difference	
	<code>*= /= %=</code>	Assignment by product, quotient, and remainder	
	<code><<= >>=</code>	Assignment by bitwise left shift and right shift	
	<code>&= ^= =</code>	Assignment by bitwise AND, XOR, and OR	
15	<code>,</code>	Comma	Left-to-right